Razonamiento Automático

GRADO EN INGENIERÍA EN INFORMÁTICA

Sesión 02.2

Mainloop, SECTION, Joypad, funciones

2026-27

P1: Videojuego ensamblador

- Pensad un juego
- Simple = mejor
- Publicaré lista de ideas
- Ensamblador
- GBTelera (Github, VM)



Crear Minijuego

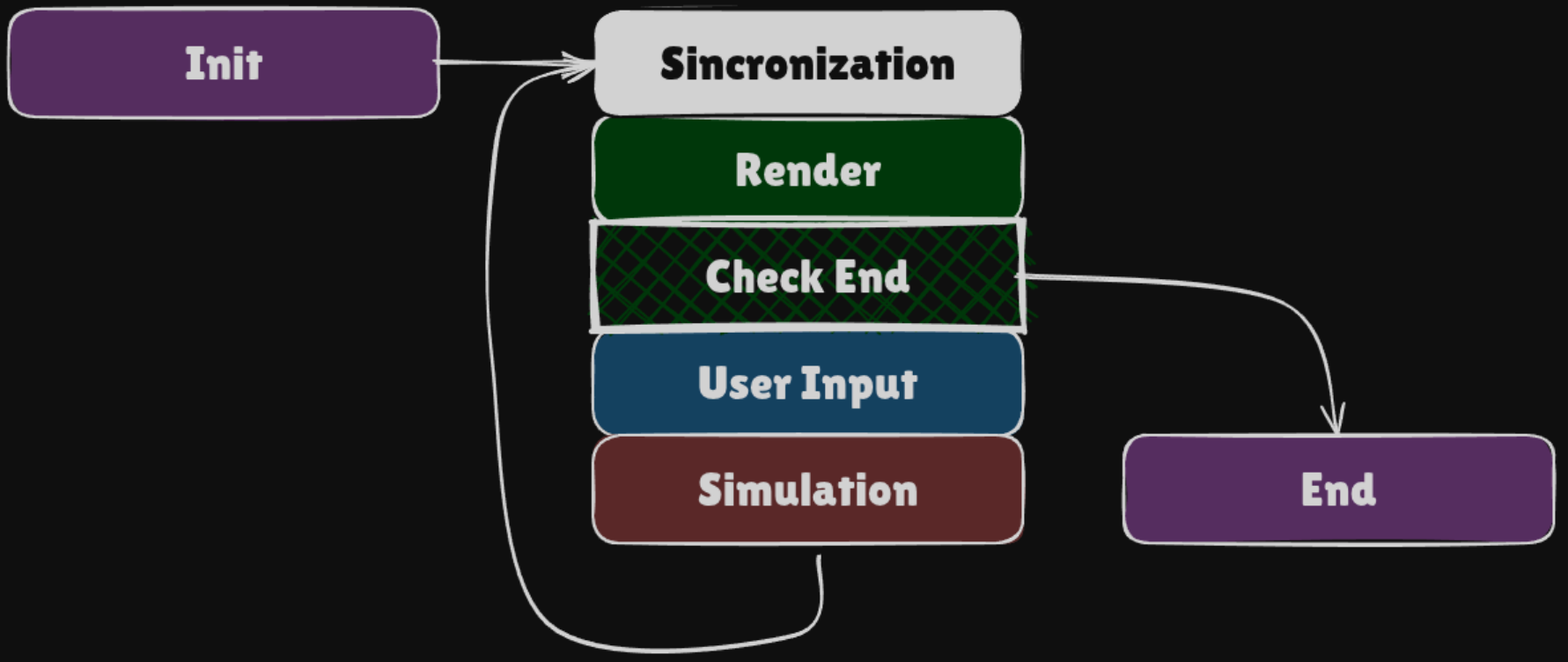
Crear Minijuego

- Debugger
- SECTIONS
- Símbolos
- Bucle principal
- Movimiento

Crear Minijuego

- Debugger
- SECTIONS
- Símbolos
- Bucle principal
- Movimiento
- RAM / Almacenar datos
- Control Joypad
- Funciones
- Gráficos - Tiles VRAM

Bucle principal



Bucle principal

```
game::
;; Init
 .mainloop:
   ;; -- wait for VBLANK --
   ;; -- do_render --

   ld a, [lives]   ;; Example
   cp 0            ;;   Game ends
   jr z, .endgame  ;;   on 0 lives

   ;; -- check_joypad --
   ;; -- simulate_game --
 jr .mainloop
 .endgame:
 ;; Do ending
```

SECTIONS

- Bloques de código o datos (individible)
- Linker los ubica

```
;; SECTION "Name", Type[Address], Options

 SECTION "Game Data", WRAM0[$C000]
 ;; Game Data

 SECTION "Entry point", ROM0[$150]
 ;; ROM Code and Data
```

Type	From	To	Size	||	Type	From	To	Size
ROM0	0000	7FFF	32K	||	WRAM0	C000	DFFF	8K
VRAM	8000	9FFF	8K	||	OAM	FE00	FE9F	160b
SRAM	A000	BFFF	8K	||	HRAM	FF80	FFFE	127b

Joypad

	Escribir	Leer	
bit	**Significado**	**Grupo 1**	**Grupo 2**
7-6	-	-	-
5	Seleccionar Grupo 1. Botones ***Select***	-	-
4	Seleccionar Grupo 2. Botones ***D-pad***	-	-
3	-	Start	Abajo
2	-	Select	Arriba
1	-	B	Izquierda
0	-	A	Derecha

$FF00 ***P1/JOYP***

```
;; Select D-Pad
ld a,%00100000;; $20
ld [$FF00],a  ;; D-Pad
```

```
;; Select Buttons
ld a,%00010000;; $10
ld [$FF00],a  ;; Buttons
```

"Funciones" (subrutinas)

⚡ Saltar y volver

¿Cómo lo hacemos?

"Funciones" (subrutinas)

⚡ Saltar y volver

¿Cómo lo hacemos?

```
;; RENDER
render:
  ;; do render
  jp render_return
;;...
;;...
.mainloop:
 ;;....
 jp render
.render_return:
 ;;....
jr .mainloop
```

"Funciones" (subrutinas)

⚡ Saltar y volver

¿Cómo lo hacemos?

```
;; RENDER
render:
  ;; do render
  jp render_return
;;...
;;...
.mainloop:
 ;;....
 jp render
.render_return:
 ;;....
jr .mainloop
```

```
check_colisions:
  ;; Check
  jp ????????
;;...
 simulation:
 ;;....
 jp check_colisions
 .check_col_return:
 ;;....
 jp check_colisions
 .check_col_return2:
 ;;....
```

"Funciones" (subrutinas)

Apuntar a dónde volver

"Funciones" (subrutinas)

Apuntar a dónde volver

```
return_address: ds 2

;; CHECK COLISIONS
check_colisions:
  ;; Check
  ;; Return to saved address
  ld a, [return_address]
  ld l, a
  ld a, [return_address+1]
  ld h, a
  jp hl
;;...
;;...
```

"Funciones" (subrutinas)

Apuntar a dónde volver

```
return_address: ds 2

;; CHECK COLISIONS
check_colisions:
  ;; Check
  ;; Return to saved address
  ld a, [return_address]
  ld l, a
  ld a, [return_address+1]
  ld h, a
  jp hl
;;...
;;...
```

```
simulation:
 ;;....
 ;; Save Return address
 ld de, .check_return_address
 ld hl, return_address
 ld [hl], e
 inc hl
 ld [hl], d

 jp check_colisions
.check_col_return:
 ;;....
```

"Funciones" (subrutinas)

Usar el mecanismo de la pila (Stack)

"Funciones" (subrutinas)

Usar el mecanismo de la pila (Stack)

```
;; RENDER
render:
    ;; Check
    ;; Return to saved address
    pop hl
    jp hl
;;...
;;...
```

"Funciones" (subrutinas)

Usar el mecanismo de la pila (Stack)

```
;; RENDER
render:
   ;; Check
   ;; Return to saved address
   pop hl
   jp hl
;;...
;;...
```

```
simulation:
 ;;....
 ;; Save Return address
  ld hl, .render_return
  push hl
  jp render
.render_return:

 ;;....
```

"Funciones" (subrutinas)

Instrucciones que implementan el mecanismo

```
;; RENDER
render:
  ;; do render
  ret
;;...
;;...
.mainloop:
 ;;....
 call render
 ;;....
jr .mainloop
```

Entender las instrucciones

Entender las instrucciones

Cada instrucción lee de/escribe en distintos "lugares"

- ***Reg*** . || Registros
- ***MEM*** || Memoria ROM o RAM
- ***Flag*** .|| Registro ***F*** = Flags

Entender las instrucciones

Cada instrucción lee de/escribe en distintos "lugares"

- ***Reg*** . || Registros
- ***MEM*** || Memoria ROM o RAM
- ***Flag*** .|| Registro ***F*** = Flags

Instrucción	Lee	Escribe
`ld a, b`	Reg(B)	Reg(A)
`cp 5`	Reg(A), MEM(5)	Reg(F):Flags(Z,C)
`add [hl]`	Reg(A), MEM(HL)	Reg(A), Reg(F):Flags(Z,C)
`bit 0, a`	Reg(A)	Reg(F):Flag(Z)

Detalles

Detalles

Valores *inmediatos* (`5`, `$0F`, `%0001`)

- Son parte de la instrucción

Detalles

Valores *inmediatos* (`5`, `$0F`, `%0001`)

- Son parte de la instrucción
- `ld a, 16` → `3E 16`

Detalles

Valores *inmediatos* (`5`, `$0F`, `%0001`)

- Son parte de la instrucción
- `ld a, 16` → `3E 16`
- Notación: `MEM($05)` o `IMM(5)`

Detalles

Valores *inmediatos* (`5`, `$0F`, `%0001`)

- Son parte de la instrucción
- `ld a, 16` → `3E 16`
- Notación: `MEM($05)` o `IMM(5)`

Reg(PC)? (con interrogante)

- Condicionales (`jr nz, ...`).

Detalles

Valores *inmediatos* (`5`, `$0F`, `%0001`)

- Son parte de la instrucción
- `ld a, 16` → `3E 16`
- Notación: `MEM($05)` o `IMM(5)`

Reg(PC)? (con interrogante)

- Condicionales (`jr nz, ...`).

Practica analizando qué se lee y qué se escribe

Razonamiento Automático

GRADO EN INGENIERÍA EN INFORMÁTICA

Sesión 02.2

Mainloop, SECTION, Joypad, funciones

2026-27